20-Day Ionic Development Intern Training Plan

Ionic Angular, TypeScript, Capacitor, device APIs, and mobile app delivery

Track: Ionic Mobile Development | Duration: 20 training days | Audience: interns who completed the shared 10-day foundation program

Purpose: build practical skills through guided learning, daily implementation, code review, and a demonstrable capstone project.

Expected outcomes

- Build a mobile-first Ionic application using Angular and TypeScript.
- Use Ionic UI components, navigation, reactive forms, and API services.
- Manage authentication state and offline-friendly behavior appropriately.
- Integrate selected native device capabilities through Capacitor plugins.
- Build, test, debug, and document an Android app with realistic device constraints.

Recommended environment

Use the team-approved Node.js LTS, Ionic CLI, Angular, Capacitor, Android Studio/SDK, Java/JDK and Gradle versions. Confirm compatibility before upgrading versions. Use a real Android device where possible; also test emulator behavior. Never commit signing keys, API secrets, tokens, or personal user data.

20-day guided curriculum

Day 1: Ionic, Angular, and TypeScript setup

Learning outcome: Run a starter app and understand its structure.

Topics / guided flow

- Ionic Angular project structure
- Angular components, templates, services
- TypeScript types and interfaces

Hands-on task: Create a starter app, run it in browser, and document the tool versions.

Reference material: Ionic Angular overview: https://ionicframework.com/docs/angular/overview | Angular tutorials: https://angular.dev/tutorials | TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html

YouTube learning: Ionic Angular beginner tutorial: https://www.youtube.com/results?search_query=Ionic+Angular+beginner+tutorial

End-of-day submission: Starter repository, version notes, and README.

Day 2: Ionic UI components and responsive layouts

Learning outcome: Build screens using Ionic components.

Topics / guided flow

- ion-header, ion-toolbar, ion-content
- Cards, lists, buttons, badges, icons
- Responsive layout and safe-area awareness

Hands-on task: Create a home screen with a list of tasks and status chips.

Reference material: Ionic UI components: https://ionicframework.com/docs/components | Ionic layout: https://ionicframework.com/docs/layout/structure

YouTube learning: Ionic components layout tutorial: https://www.youtube.com/results?search_query=Ionic+Angular+UI+components+layout+tutorial

End-of-day submission: Home screen and component inventory.

Day 3: Angular services and dependency injection

Learning outcome: Separate view code from business logic.

Topics / guided flow

- Services and dependency injection
- Interfaces and typed models
- Observable basics

Hands-on task: Move mock task data and list filtering into a service.

Reference material: Angular services: https://angular.dev/guide/di | Angular essentials: https://angular.dev/essentials

YouTube learning: Angular services dependency injection tutorial: https://www.youtube.com/results?search_query=Angular+services+dependency+injection+tutorial

End-of-day submission: Typed model, service, and working list.

Day 4: Navigation and page lifecycle

Learning outcome: Build predictable navigation between screens.

Topics / guided flow

- Angular Router and route parameters
- Ionic router outlet and navigation
- Page lifecycle hooks and refresh behavior

Hands-on task: Create list, detail, and settings pages with navigation.

Reference material: Ionic navigation: https://ionicframework.com/docs/angular/navigation | Angular routing: https://angular.dev/guide/routing

YouTube learning: Ionic Angular navigation router tutorial: https://www.youtube.com/results?search_query=Ionic+Angular+navigation+router+tutorial

End-of-day submission: Three-page app with route parameters.

Day 5: Reactive forms and validation

Learning outcome: Build a robust data-entry form.

Topics / guided flow

- Reactive forms and validators
- Input feedback and accessibility
- Submit, cancel, reset, and error states

Hands-on task: Build a task/visit form with required fields and clear validation.

Reference material: Angular forms: https://angular.dev/guide/forms | Ionic input: https://ionicframework.com/docs/api/input

YouTube learning: Ionic Angular reactive forms validation: https://www.youtube.com/results?search_query=Ionic+Angular+reactive+forms+validation

End-of-day submission: Validated form and manual test checklist.

Day 6: HTTP and REST API integration

Learning outcome: Connect the app to a backend API.

Topics / guided flow

- HttpClient and typed responses
- Base URL by environment
- Loading, retry, empty, and error states

Hands-on task: Fetch a list from a mock or approved development API.

Reference material: Angular HTTP: https://angular.dev/guide/http | Ionic Native HTTP overview: https://ionicframework.com/docs/native

YouTube learning: Ionic Angular REST API integration: https://www.youtube.com/results?search_query=Ionic+Angular+REST+API+integration+HttpClient

End-of-day submission: API service, environment configuration, and UI states.

Day 7: Authentication and guarded navigation

Learning outcome: Implement a safe authentication flow.

Topics / guided flow

- Login/logout UX
- Token storage trade-offs
- Route guards and backend authorization

Hands-on task: Build a login flow against a development endpoint or mock service; do not store secrets in source.

Reference material: Angular route guards: https://angular.dev/guide/routing/route-guards | OWASP MASVS: https://mas.owasp.org/MASVS/

YouTube learning: Ionic Angular login authentication tutorial: https://www.youtube.com/results?search_query=Ionic+Angular+login+authentication+tutorial

End-of-day submission: Auth flow diagram and protected-screen behavior.

Day 8: App state and reusable patterns

Learning outcome: Keep shared state understandable.

Topics / guided flow

- Observable/shared service patterns
- Avoid duplicate API calls
- Refresh after navigation and logout

Hands-on task: Implement a small shared state service for the logged-in user and task list.

Reference material: RxJS: https://rxjs.dev/guide/overview | Angular signals: https://angular.dev/guide/signals

YouTube learning: Angular state management services RxJS: https://www.youtube.com/results?search_query=Angular+state+management+service+RxJS

End-of-day submission: Shared state service and tests/manual scenarios.

Day 9: Offline and unreliable network handling

Learning outcome: Design for intermittent connectivity.

Topics / guided flow

- Network failure UX
- Retry and safe resubmission
- Local cache and stale-data labels

Hands-on task: Add offline/error banners and prevent accidental duplicate submissions.

Reference material: Capacitor Network: https://capacitorjs.com/docs/apis/network | MDN offline concepts: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Offline_Service_workers

YouTube learning: Ionic offline handling network status: https://www.youtube.com/results?search_query=Ionic+Capacitor+offline+network+handling

End-of-day submission: Network state handling and offline test notes.

Day 10: Mobile UX checkpoint

Learning outcome: Polish a small mobile workflow.

Topics / guided flow

- Touch targets and readable text
- Keyboard behavior and scrolling
- Loading and empty states

Hands-on task: Demo login, list, detail, and form flow on a phone or emulator.

Reference material: Ionic accessibility: https://ionicframework.com/docs/developing/accessibility | Material accessibility: https://m3.material.io/foundations/accessible-design/overview

YouTube learning: Ionic mobile UI UX best practices: https://www.youtube.com/results?search_query=Ionic+mobile+UI+UX+best+practices

End-of-day submission: Mini-project demo and mentor review.

Day 11: Capacitor fundamentals

Learning outcome: Understand web-to-native app packaging.

Topics / guided flow

- Capacitor project structure
- Syncing web assets and native projects
- Plugin install/configure workflow

Hands-on task: Add or inspect Android platform and run a development build.

Reference material: Capacitor docs: https://capacitorjs.com/docs | Android setup: https://capacitorjs.com/docs/android

YouTube learning: Ionic Capacitor Android setup tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+Android+setup+tutorial

End-of-day submission: Successful Android build and documented commands.

Day 12: Camera and media capture

Learning outcome: Use device media capability with permission handling.

Topics / guided flow

- Camera plugin and permission prompts
- Photo URI versus base64 trade-offs
- Preview, cancel, retry, and upload

Hands-on task: Capture or select a photo and display a preview before upload.

Reference material: Capacitor Camera: https://capacitorjs.com/docs/apis/camera | Ionic file input: https://ionicframework.com/docs/api/input

YouTube learning: Ionic Capacitor camera tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+Camera+plugin+tutorial

End-of-day submission: Photo capture flow and permission test cases.

Day 13: Location and geolocation

Learning outcome: Request location only when the feature needs it.

Topics / guided flow

- Permission states and denied permissions
- Accuracy, timeout, and failure handling
- Privacy-conscious location UX

Hands-on task: Build a location-check screen with clear permission messaging and a fallback.

Reference material: Capacitor Geolocation: https://capacitorjs.com/docs/apis/geolocation | Android location permissions: https://developer.android.com/develop/sensors-and-location/location/permissions

YouTube learning: Ionic Capacitor geolocation tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+geolocation+tutorial

End-of-day submission: Location screen and permission/failure scenarios.

Day 14: Local storage and secure data handling

Learning outcome: Persist only the data the app actually needs.

Topics / guided flow

- Preferences versus database/cache
- Token handling and logout cleanup
- Data expiry and privacy considerations

Hands-on task: Persist a non-sensitive preference and implement logout cleanup.

Reference material: Capacitor Preferences: https://capacitorjs.com/docs/apis/preferences | OWASP MASVS storage: https://mas.owasp.org/MASVS/

YouTube learning: Ionic Capacitor Preferences storage tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+Preferences+storage+tutorial

End-of-day submission: Storage service, cleanup behavior, and notes.

Day 15: Background tasks and push notifications concepts

Learning outcome: Understand mobile lifecycle limits and notification flow.

Topics / guided flow

- Foreground/background lifecycle
- Push token registration concepts
- Backend ownership and permission prompts

Hands-on task: Document a push-notification flow; implement only if the project and backend are approved.

Reference material: Capacitor Push Notifications: https://capacitorjs.com/docs/apis/push-notifications | Android app lifecycle: https://developer.android.com/guide/components/activities/activity-lifecycle

YouTube learning: Ionic Capacitor push notifications tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+push+notifications+tutorial

End-of-day submission: Sequence diagram and optional working notification test.

Day 16: Performance and device debugging

Learning outcome: Find slow screens and device-specific issues.

Topics / guided flow

- Chrome remote debugging and Android logs
- Large images, change detection, repeated calls
- Low-memory and low-storage scenarios

Hands-on task: Profile one slow workflow and implement one measured improvement.

Reference material: Chrome DevTools: https://developer.chrome.com/docs/devtools/ | Android debugging: https://developer.android.com/studio/debug

YouTube learning: debug Ionic Android app Chrome DevTools: https://www.youtube.com/results?search_query=debug+Ionic+Android+app+Chrome+DevTools

End-of-day submission: Before/after notes and device test results.

Day 17: Testing and release quality

Learning outcome: Test the app beyond the happy path.

Topics / guided flow

- Unit tests for services and utilities
- Manual test matrix across devices
- API failure, permission denial, offline cases

Hands-on task: Create a test matrix and automate at least a few service/component tests.

Reference material: Angular testing: https://angular.dev/guide/testing | Capacitor testing: https://capacitorjs.com/docs/guides/testing

YouTube learning: Ionic Angular unit testing tutorial: https://www.youtube.com/results?search_query=Ionic+Angular+unit+testing+tutorial

End-of-day submission: Test matrix, test results, and known issues.

Day 18: Android build, signing, and release basics

Learning outcome: Prepare a repeatable debug/release build process.

Topics / guided flow

- Build and sync commands
- App ID, version code/name, icons and splash

- Signing key protection and build variants

Hands-on task: Produce a debug APK for internal testing and document build steps.

Reference material: Capacitor Android deployment: https://capacitorjs.com/docs/android/deployment | Android app signing: https://developer.android.com/studio/publish/app-signing

YouTube learning: Ionic Android APK build tutorial: https://www.youtube.com/results?search_query=Ionic+Capacitor+build+Android+APK+tutorial

End-of-day submission: APK or build evidence, version details, and steps.

Day 19: Capstone integration and documentation

Learning outcome: Integrate the app's core workflow and harden it.

Topics / guided flow

- End-to-end user flow
- Permissions, errors, offline behavior
- README and API contracts

Hands-on task: Finish a field-visit, patrol, checklist, or asset workflow using approved APIs.

Reference material: Ionic deployment: https://ionicframework.com/docs/deployment | OWASP MASVS: https://mas.owasp.org/MASVS/

YouTube learning: Ionic Angular CRUD mobile app project: https://www.youtube.com/results?search_query=Ionic+Angular+CRUD+mobile+app+project

End-of-day submission: Integrated capstone, README, and test evidence.

Day 20: Final demo and mobile QA review

Learning outcome: Demonstrate a reliable app and explain device trade-offs.

Topics / guided flow

- Live demo on emulator/device
- Explain permission and offline behavior
- Bug triage and future improvements

Hands-on task: Run the app on a real device if available; show one failure case and recovery.

Reference material: Android quality guidelines: https://developer.android.com/docs/quality-guidelines | README guide: https://www.makeareadme.com/

YouTube learning: how to present an Ionic mobile app project: https://www.youtube.com/results?search_query=how+to+present+an+Ionic+mobile+app+project

End-of-day submission: Demo, source code, build steps, test matrix, and reflection.

Mentor checkpoints

- Day 5: review fundamentals, repository structure, and ability to explain the work without reading notes.
- Day 10: review the first integrated mini-project and give specific corrective feedback.
- Day 15: review advanced implementation, tests, and handling of edge cases.
- Day 20: live demo, code walkthrough, README, known limitations, and individual reflection.

Capstone project

Build a field-operations app with login, task/visit list, detail screen, validated form, photo capture, optional location check, API integration, offline/error handling, and Android build instructions. Use mock data or approved development APIs only. Confirm plugin and Android toolchain compatibility before adding native plugins.

Assessment rubric (100 points)

Area	Points
UI, Angular, and TypeScript fundamentals	20
API integration and app state	20
Capacitor/device capability implementation	20
Reliability, testing, and Android build	25
Communication and documentation	15

Daily working agreement

- Start with a short plan; spend time implementing rather than only watching tutorials.
- Commit code daily with clear messages. Never commit API keys, passwords, personal data, or production credentials.
- Use official documentation first; use videos to reinforce concepts, not as a substitute for implementation.
- Ask for help with a reproducible example, error message, what you tried, and what you expected.
- At the end of each day, update README or learning notes with what works, what failed, and the next step.

Link note: YouTube links are targeted search links so mentors can select an up-to-date, suitable tutorial. Prefer recent videos that match the tool versions used by the team.